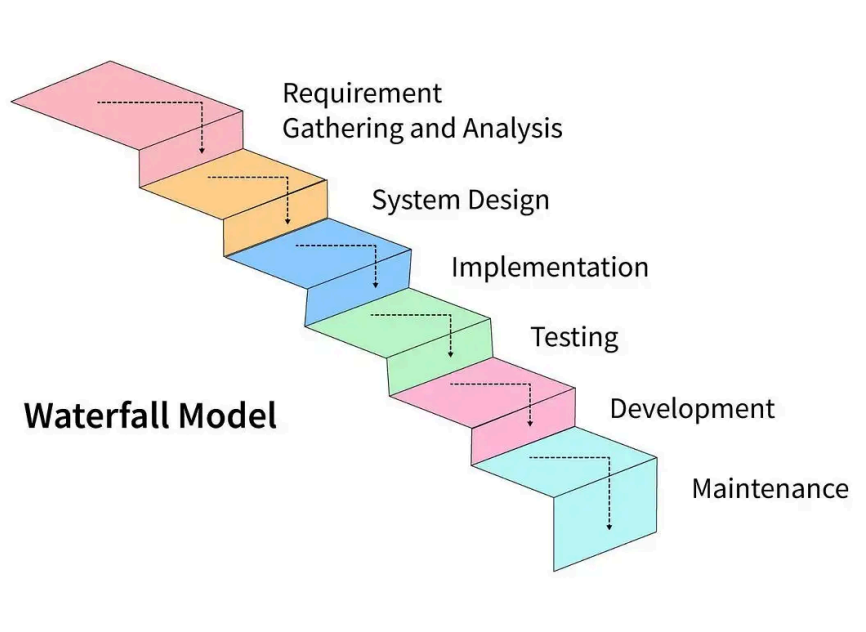
# DevOps

## Brief history of software production models

The software production lifecycle consists of three essential stages, each playing a critical role in delivering a high-quality product that meets client expectations and performs reliably in a live environment:

1. **Requirements Gathering**
   - Define the scope, functionality, and specifications of the software.
   - Document requirements to guide the development process.
2. **Development and Testing**
   - Design and implement the software according to the requirements.
   - Perform testing to identify and resolve defects.
3. **Operations and Infrastructure**
   - Set up and manage the infrastructure required for the application.
   - Monitor, maintain, and update the system to ensure performance, security, and reliability.

### The Waterfall model

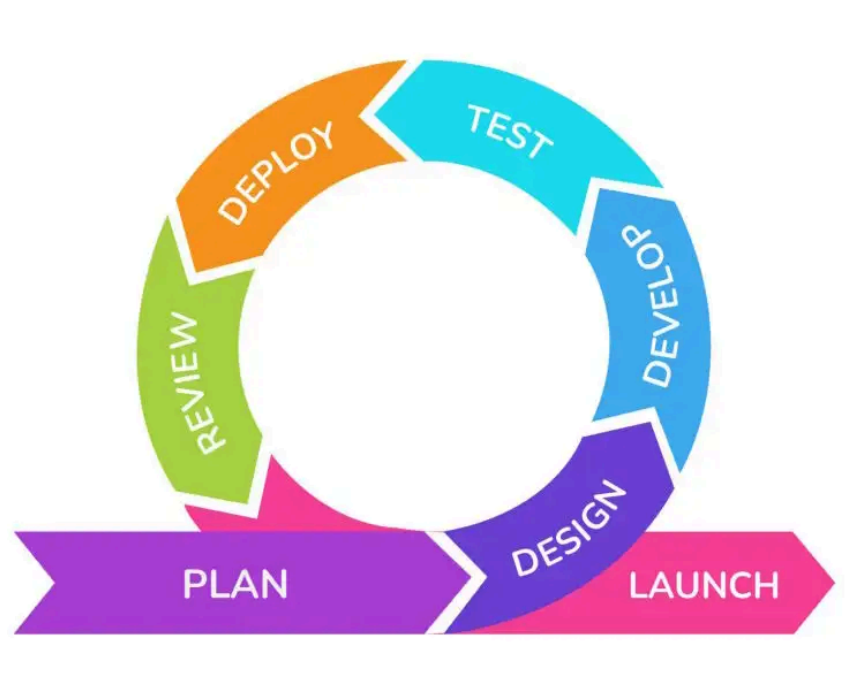

Requirement Gathering and Analysis
System Design
Implementation
Testing
Development
Maintenance
Waterfall Model

The waterfall model creates friction between (1) requirements gathering and (2) development due to its linear and rigid structure, where each phase must be completed before moving to the next.

As development progresses, teams frequently discover **gaps, ambiguities, or misunderstandings** in the initial requirements, leading to delays, costly changes, or project misalignment.

- **Rigid Change Management**: Adapting to new requirements is difficult and costly.
- **Late Discovery of Issues**: Problems in requirements often surface deep into development.
- **Misalignment with Client Expectations**: What is delivered may no longer match what the client actually needs.
- **Long Feedback Loops**: Clients only see the final product, limiting opportunities for early correction.

### The Agile model

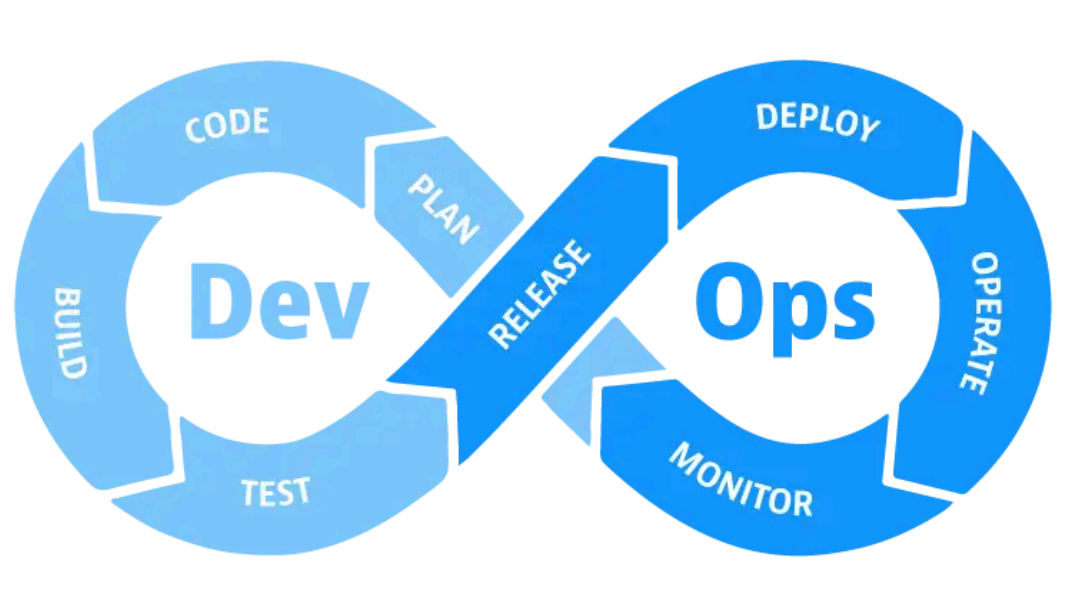

DEPLOY
TEST
DEVELOP
DESIGN
REVIEW
PLAN
LAUNCH

The Agile model (Kanban (1950s), Agile Manifesto (2001)) eases the friction between (1) requirements gathering and (2) development by adopting an iterative and flexible approach to software production. Instead of trying to define all requirements upfront, Agile promotes collaboration and continuous feedback throughout the development lifecycle. Requirements are gathered incrementally and revisited during each iteration, allowing for adjustments based on client feedback, changing needs, or new insights.

However, **it neglects the operational aspects of software production** leading to friction between (2) development and (3) operations:

- **Limited Integration with Operations**: Dev teams might build features without thinking about how they'll be deployed, scaled, or maintained.
- **Deployment as an Afterthought**: Shipping to production happens late in the cycle and often causes problems.
- **Low Feedback**: Once the software is live, there's weak feedback from production issues back into the development cycle.

## The DevOps model


CODE
PLAN
BUILD
Dev
TEST
RELEASE
DEPLOY
Ops
OPERATE
MONITOR

- DevOps is a collaborative and multidisciplinary organizational effort to automate the continuous delivery of new software updates while ensuring their correctness and reliability. (Leite et al., 2020)
- **Teams integrate into a single unit (*"you build it, you run it"*)**, where developers participate throughout the entire application lifecycle, from development to production.
- Teams use **automation** to accelerate manual processes.
- DevOps practices are **highly beneficial in managing distributed systems**, which are inherently difficult to operate manually.

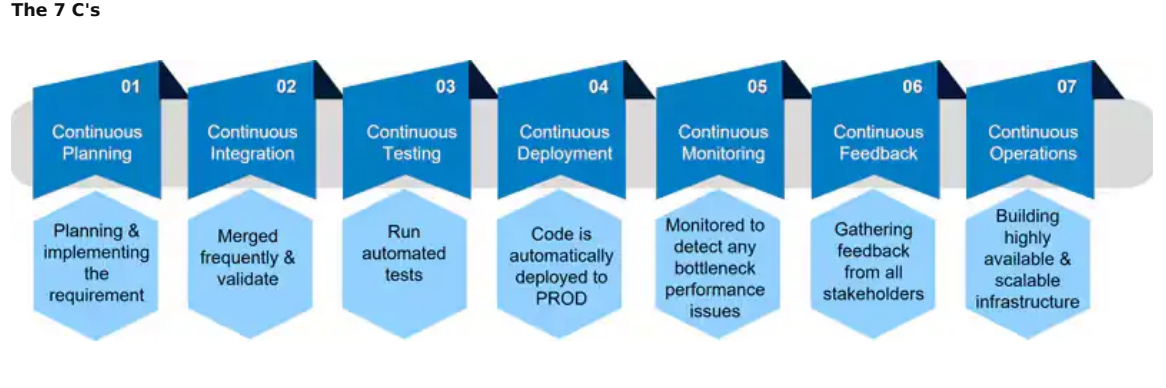
**What implies for Developers:**

- **When something breaks, developers are called to fix it**. This encourages developers to write more robust code, increase test coverage, enhance observability etc.
- Automation allows fewer people to manage more code and handle increased complexity, but this can create a more stressful environment as the **responsibility scales**.

> *"The traditional model is that you take your software to the wall that separates development and operations and throw it over and then forget about it. Not at Amazon. You build it, you run it. This brings developers into contact with the day-to-day operation of their software. It also brings them into day-to-day contact with the customer. This customer feedback loop is essential for improving the quality of the service." - Amazon CTO Werner Vogels (2006)*

> *"Something that I should have known all along became clearer and clearer to me: Methodology and tools will not change your organisation. They can support it but culture is the important ingredient that was missing. As Drucker says: "Culture eats strategy for breakfast". It is so very true … But how do I really change the culture of an organisation, how do I avoid the old saying that "to change people, you sometimes have to change (read replace) people" ? - Mirco Hering (DevOps Engineer)"*

**The 7 C's**


01
Continuous Planning
Planning & implementing the requirement
02
Continuous Integration
Merged frequently & validate
03
Continuous Testing
Run automated tests
04
Continuous Deployment
Code is automatically deployed to PROD
05
Continuous Monitoring
Monitored to detect any bottleneck performance issues
06
Continuous Feedback
Gathering feedback from all stakeholders
07
Continuous Operations
Building highly available & scalable infrastructure

1. **Continuous Planning**
   This phase involves planning and developing the software. Development is broken into smaller tasks, following Agile methodologies that focus on "just-in-time" requirements. It includes defining user stories and refining the product backlog.
2. **Continuous Integration**
   In this phase, developers write code and push changes to the source code management system. All code changes are merged frequently and validated through automated builds, tests, and security checks. This helps the team identify issues early in the development process. **Continuous Delivery** refers to the continuous creation of an updated software artifact (e.g., mvn package/docker build).
3. **Continuous Testing**
   This phase involves writing and running automated test cases using various tools such as Selenium, JUnit, and TestNG. Automated tests help ensure that the software is functioning correctly and meets quality standards.
4. **Continuous Deployment**
   In this phase, application code is automatically deployed to production environments, facilitating seamless delivery. This ensures that new features and fixes are rapidly available to users.
5. **Continuous Monitoring**
   This phase ensures that systems and applications are continuously monitored to detect any bottlenecks, performance issues, or downtime. Alerts can be set to notify stakeholders, allowing for timely action to resolve any issues.
6. **Continuous Feedback**
   This crucial phase involves gathering feedback from all stakeholders, including developers, testers, product owners, and end users. This feedback helps the team identify what went well and what can be improved, driving continuous improvement in the development process.
7. **Continuous Operations**
   This phase ensures that systems are available 24/7. It focuses on building highly available and scalable infrastructure through automation to minimize downtime and ensure reliability.

## General concepts

### Domain Specific Language (DSL)

Domain-Specific Languages (DSLs) are widely used in CI/CD to simplify automation and pipeline management. These DSLs are designed to handle specific tasks in the CI/CD process, making it easier to define, manage, and maintain workflows.

| Aspect | Declarative DSL | Imperative DSL |
|---|---|---|
| **Definition** | Describes *what* needs to be done | Describes *how* tasks should be performed |
| **Examples** | YAML | Groovy, Bash |
| **Ease of Use** | Simple, easy to configure, more abstract | Greater control, flexible but more complex |
| **Control** | Limited control over execution flow | Full control over logic and task execution |
| **Use Cases** | Standardized workflows, repeatable tasks | Custom, complex tasks and workflows |
| **Maintenance** | Easier to maintain, less code to write | Harder to maintain due to complexity |

YAML (Declarative)

```
name: Java CI

on:
  push:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout source
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          java-version: '17'

      - name: Build project
        run: mvn clean package
```

Groovy (Imperative)

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                script {
                    if (env.BRANCH_NAME == 'main') {
                        sh 'mvn clean package'
                    } else {
                        sh 'mvn clean compile'
                    }
                }
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }
    }
}
```

### Infrastructure as Code (IaC)

**Infrastructure as Code (IaC)** is the practice of **managing and provisioning infrastructure through machine-readable definition files** — rather than through manual processes.

- **Declarative or Imperative**: You can describe the *desired end state* (declarative) or the *steps to reach it* (imperative).
- **Version-controlled**: Infrastructure code is stored in systems like Git for traceability, review, and rollback.
- **Repeatable**: The same code can be used to create identical environments.
- **Automated**: Tools execute code to provision, update, or destroy resources automatically.

### Configuration as Code (CaC)

Configuration drift happens when a system's actual state diverges from its intended configuration.

**What causes configuration drift?**

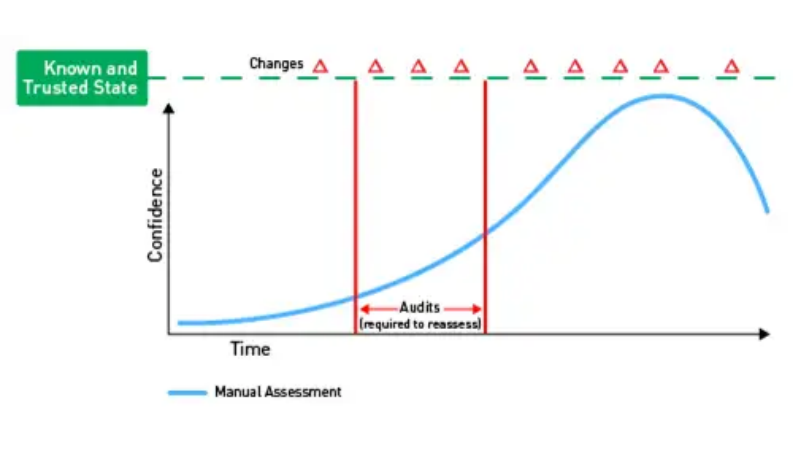
- Manual changes made directly on systems (e.g., via SSH)
- Failed or partial deployments
- Unmanaged updates or patches
- Lack of version control in configuration files

**Why is configuration drift a problem?**

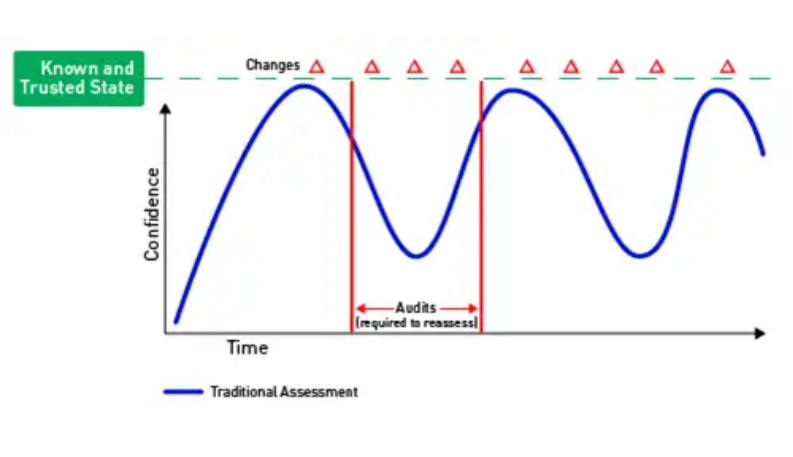
- Leads to **inconsistencies** across servers
- Makes **debugging and troubleshooting** harder
- Violates **security baselines**, audit requirements
- Can lead to **compliance** drift

**How to detect configuration drift?**

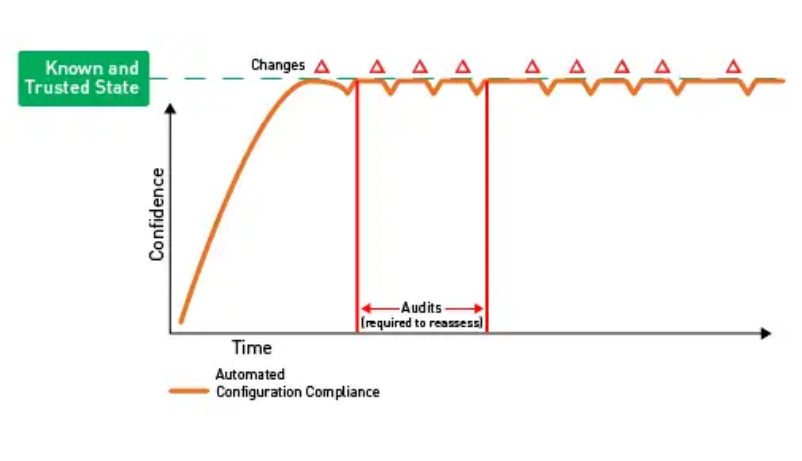
1. Manually monitor. This is incredibly time-consuming and therefore is not done on a regular basis, if at all.



2. Scan for compliance. While not as tedious as the first level, this still requires a certain level of interaction to create administrative credentials for the tool to scan with, as well as someone to schedule or run the scans when required and remediate the results. This is typically done once a month or once a quarter to try to get ahead of the audit process.



3. Monitor all systems in a near real-time manner. This would require that the systems are provisioned with a lightweight agent that can monitor/update the systems continuously.



### GitOps

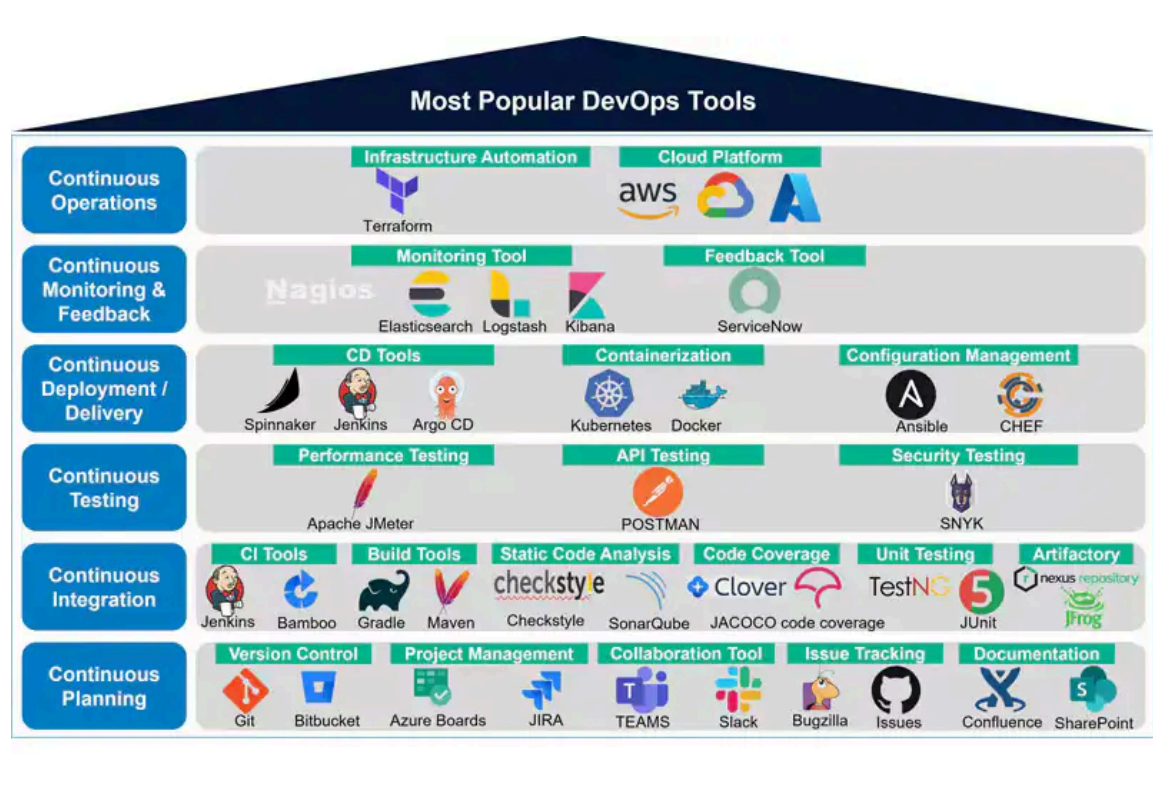
**GitOps** is a DevOps methodology that uses **Git as the single source of truth** for defining and managing **infrastructure and application configurations**.

With GitOps, all changes to your infrastructure or applications are made by **modifying Git repositories**, and then **automated systems** apply those changes to the runtime environment.

Key Principles:

- **Declarative Infrastructure**: Your system's desired state is described declaratively.
- **Version-controlled**: Infrastructure code is stored in systems like Git for traceability, review, and rollback.
- **Automated Delivery**: Agents or CI/CD pipelines detect changes in Git and automatically apply them.
- **Continuous Reconciliation**: The actual state of the system is continuously compared to the desired state in Git, and drift is corrected.
  - Chef InSpec
  - Ansible lint
  - Terraform DriftCTL

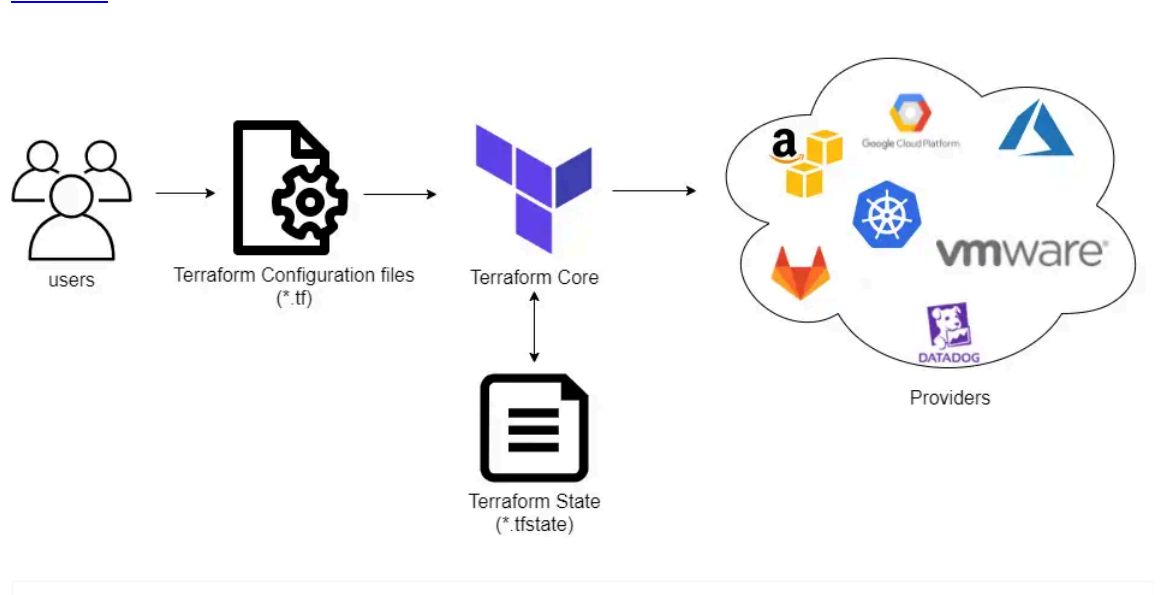
## DevOps Pipelines Overview


Most Popular DevOps Tools
Continuous Operations
Infrastructure Automation
Terraform
Cloud Platform
aws
Continuous Monitoring & Feedback
Monitoring Tool
Nagios
Elasticsearch
Logstash
Kibana
Feedback Tool
ServiceNow
Continuous Deployment / Delivery
CD Tools
Spinnaker
Jenkins
Argo CD
Containerization
Kubernetes
Docker
Configuration Management
Ansible
CHEF
Continuous Testing
Performance Testing
Apache JMeter
API Testing
POSTMAN
Security Testing
SNYK
Continuous Integration
CI Tools
Jenkins
Bamboo
Gradle
Maven
Static Code Analysis
checkstyle
Checkstyle
SonarQube
Code Coverage
Clover
JACOCO code coverage
Unit Testing
TestNG
JUnit
Artifactory
nexus repository
JFrog
Continuous Planning
Version Control
Git
Bitbucket
Project Management
Azure Boards
JIRA
Collaboration Tool
TEAMS
Slack
Issue Tracking
Bugzilla
Issues
Documentation
Confluence
SharePoint

### Infrastructure Provisioning

- **What it does**: Provision cloud infrastructure (EC2, VPCs, load balancers, etc.) using declarative configuration (Infrastructure As Code).
- **Use case**: You define resources in `.tf` files and apply them via `terraform apply`.

Terraform

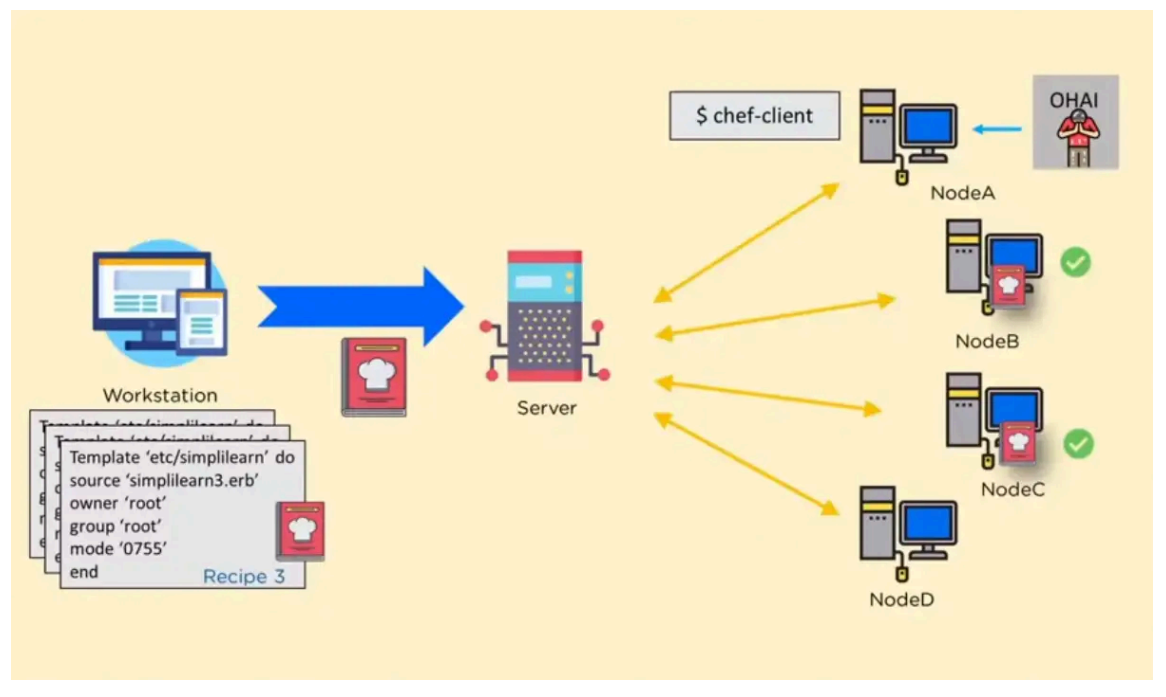

users
Terraform Configuration files (*.tf)
Terraform Core
Terraform State (*.tfstate)
Google Cloud Platform
vmware
DATADOG
Providers

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "example" {
  bucket = "my-unique-example-bucket-123456"
  acl    = "private"

  tags = {
    Name        = "MyExampleBucket"
    Environment = "Dev"
  }
}

resource "aws_instance" "web" {
  ami           = "ami-0c94855ba95c71c99"  # Example Amazon Linux 2 AMI in us-east-1
  instance_type = "t2.micro"

  tags = {
    Name = "MyWebServer"
  }
}
```

### Configuration Management

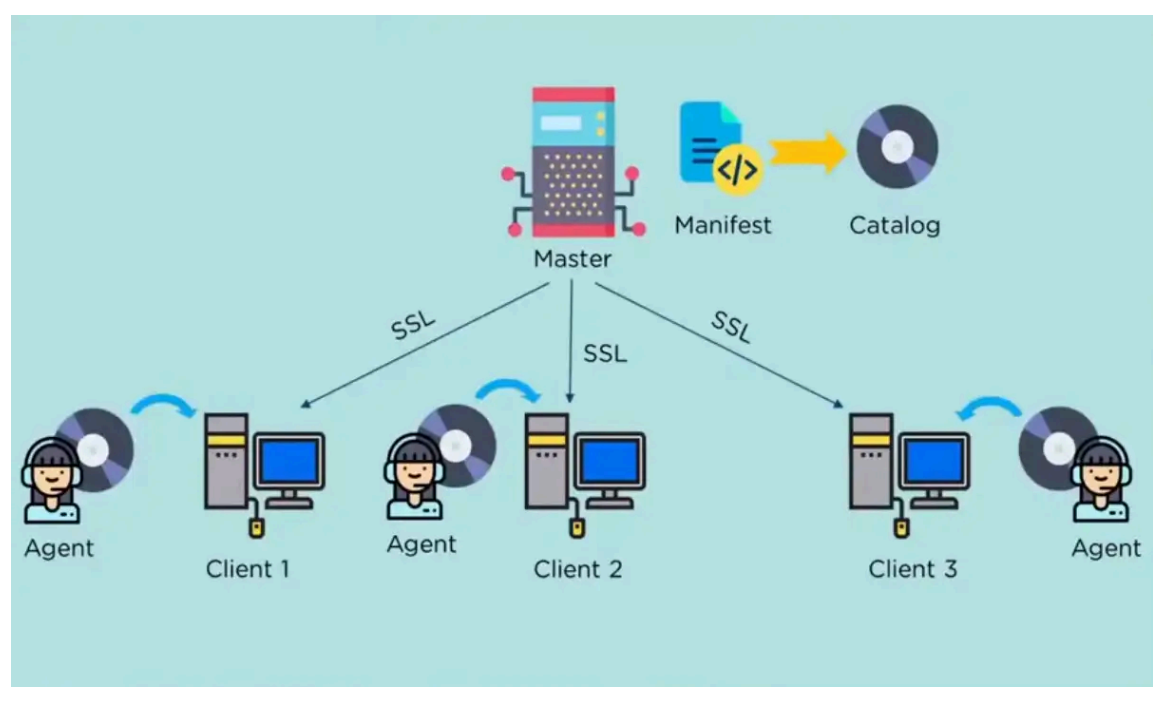
- **What it does**: Tools based on textual configuration files (recipes, playbooks, etc.) which define automation tasks (installing software, configuring services, copying files, or restarting servers) to be executed on remote systems.
- **Use case**: Install Apache, set environment variables, deploy files.

Chef (Master/Agent, Imperative, Ruby DSL, Hard to learn).


$ chef-client
OHAI
NodeA
NodeB
NodeC
NodeD
Server
Workstation
Template 'etc/simplilearn' do
source 'simplilearn3.erb'
owner 'root'
group 'root'
mode '0755'
end
Recipe 3

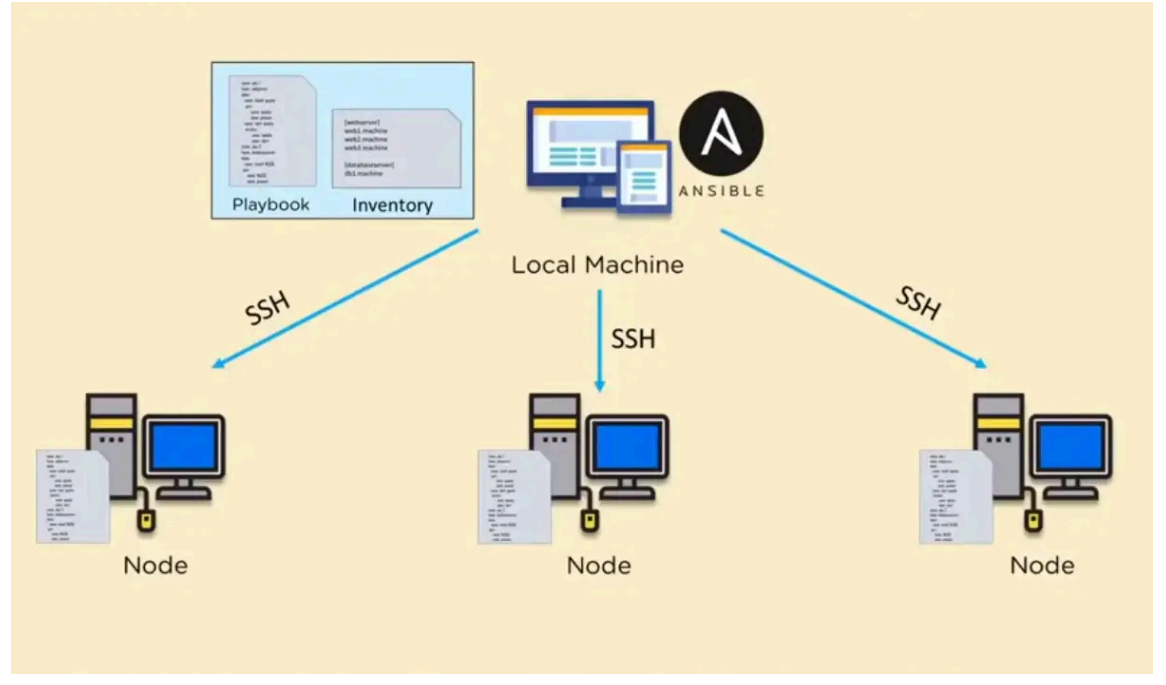
```
# Install Apache package
package 'httpd' do
  action :install
end

# Start and enable the Apache service
service 'httpd' do
  action [:enable, :start]
end

# Create a custom homepage
file '/var/www/html/index.html' do
  content '<h1>Welcome to My Web Server!</h1>'
  mode '0644'
  owner 'root'
  group 'root'
end
```

Puppet: (Master/Agent, Declarative, Ruby DSL, Hard to learn)


Master
Manifest
Catalog
SSL
SSL
SSL
Agent
Client 1
Agent
Client 2
Client 3
Agent

```
# Install Apache package
package { 'httpd':
  ensure => installed,
}

# Enable and start the Apache service
service { 'httpd':
  ensure     => running,
  enable     => true,
  require    => Package['httpd'],
}

# Create a custom homepage
file { '/var/www/html/index.html':
  ensure  => file,
  content => '<h1>Welcome to My Web Server!</h1>',
  mode    => '0644',
  owner   => 'root',
  group   => 'root',
  require => Package['httpd'],
}
```

Ansible: (Agentless, Wannabe Declarative, YAML DSL, Easy to learn)


Playbook
Inventory
ANSIBLE
Local Machine
SSH
SSH
SSH
Node
Node
Node

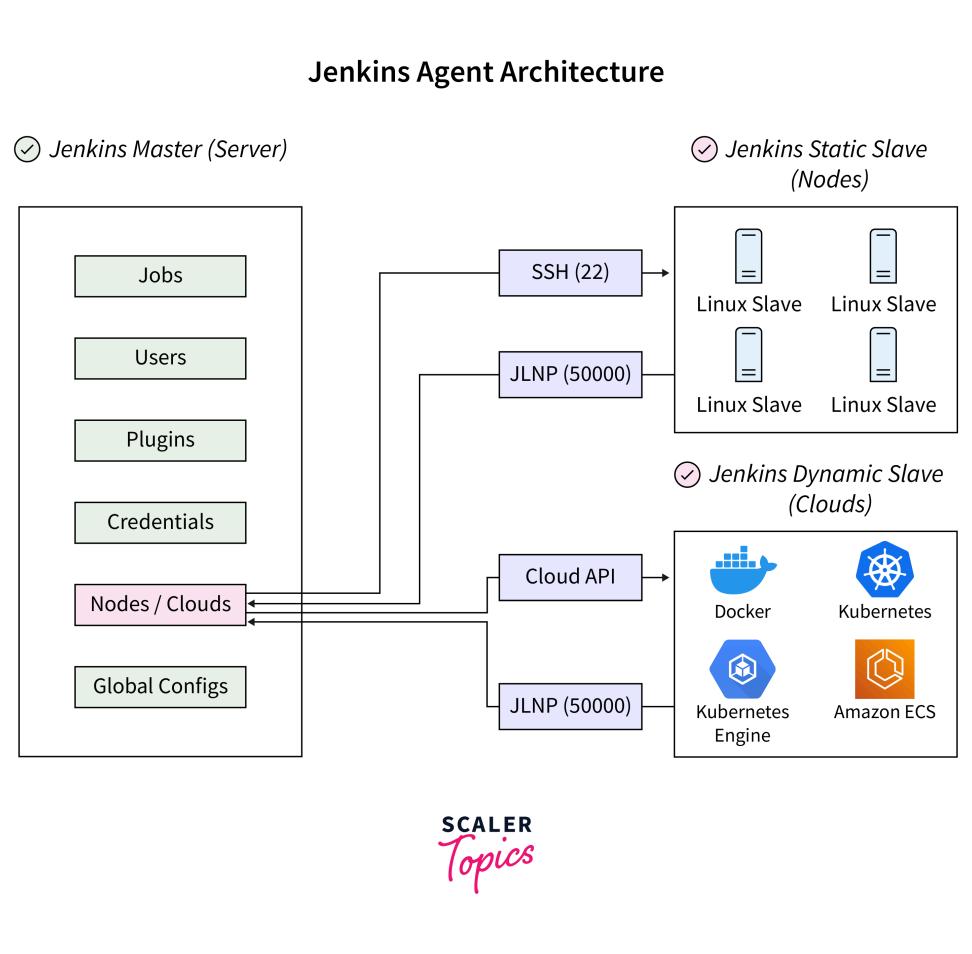
```
- name: Setup Apache on web servers
  hosts: webservers
  become: true

  tasks:
    - name: Install Apache
      package:
        name: httpd
        state: present

    - name: Start and enable Apache service
      service:
        name: httpd
        state: started
        enabled: yes

    - name: Create custom homepage
      copy:
        dest: /var/www/html/index.html
        content: "<h1>Welcome to My Web Server!</h1>"
        owner: root
        group: root
        mode: '0644'
```

---

### Build, Test, Deployment Automation

- **What it does**: Automates build, test, package, deploy steps using pipelines.
- **Use case**:
  1. Triggered by a `git push`.
  2. Runs `git pull/clone`, `mvn clean`, `mvn test`, `mvn package`, `docker build`, `docker push`.

Jenkins: (Master/Agent, Imperative, Groovy DSL)


Jenkins Agent Architecture
Jenkins Master (Server)
Jobs
Users
Plugins
Credentials
Nodes / Clouds
Global Configs
SSH (22)
JLNP (50000)
Cloud API
JLNP (50000)
Jenkins Static Slave (Nodes)
Linux Slave
Linux Slave
Linux Slave
Linux Slave
Jenkins Dynamic Slave (Clouds)
Docker
Kubernetes
Kubernetes Engine
Amazon ECS
SCALER Topics

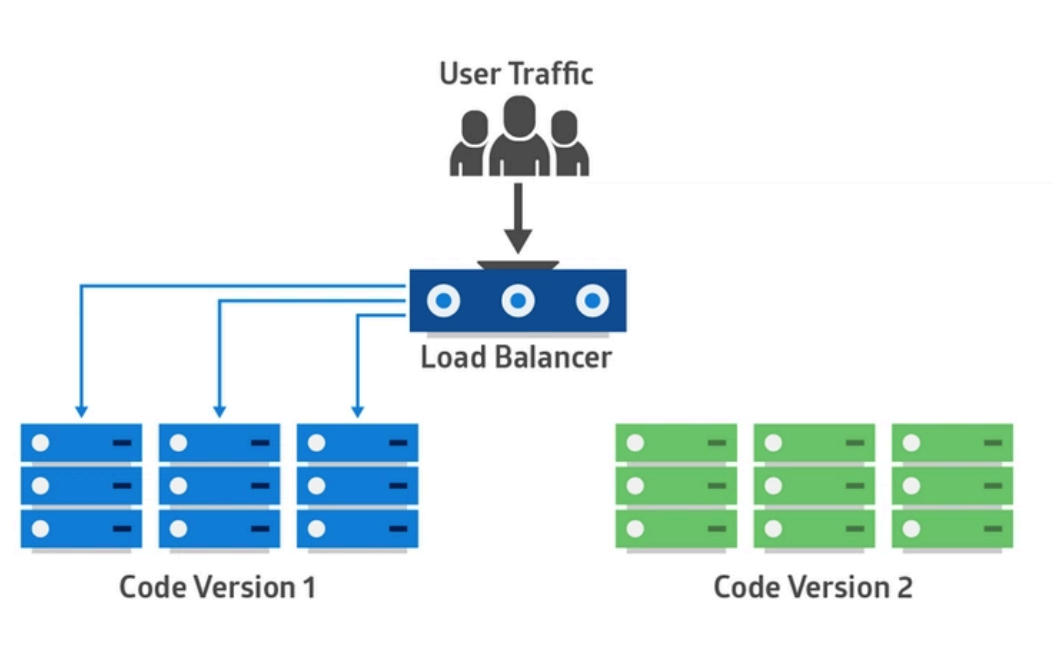
```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = 'yourdockerhubusername/springboot-app'
        DOCKER_CREDENTIALS_ID = 'docker-hub-credentials'
    }

    stages {
        stage('Checkout') {
            steps {
                git clone 'https://github.com/your-org/your-java-service.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests=false'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Build Docker Image') {
            steps {
                script {
                    dockerImage = docker.build("${DOCKER_IMAGE}:${env.BUILD_NUMBER}")
                }
            }
        }

        stage('Push to Docker Hub') {
            steps {
                script {
                    docker.withRegistry('https://index.docker.io/v1/', DOCKER_CREDENTIALS_ID) {
                        dockerImage.push('latest')
                    }
                }
            }
        }
    }

    post {
        always {
            echo "Cleaning up..."
            sh 'docker image prune -f'
        }
        success {
            echo "Pipeline completed successfully!"
        }
        failure {
            echo "Pipeline failed!"
        }
    }
}
```

GitHub Actions: (Cloud-based ephemeral runners, YAML, Imperative)

```
name: Build and Push Spring Boot App

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    env:
      DOCKER_IMAGE: yourdockerhubusername/springboot-app

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build with Maven
        run: mvn clean package -DskipTests=false

      - name: Run tests
        run: mvn test

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ${{ env.DOCKER_IMAGE }}:${{ github.run_number }}
            ${{ env.DOCKER_IMAGE }}:latest

      - name: Clean up Docker
        run: docker system prune -af
```

## Deployment Strategies

**Blue-Green Deployment** uses two identical environments—one active (blue) and one idle (green). The new version is deployed to the idle environment and tested. Once verified, all traffic is instantly switched to it.


User Traffic
Load Balancer
Code Version 1
Code Version 2

When to choose it

- You need **zero downtime**
- You want **instant rollback**
- You can afford **double infrastructure**

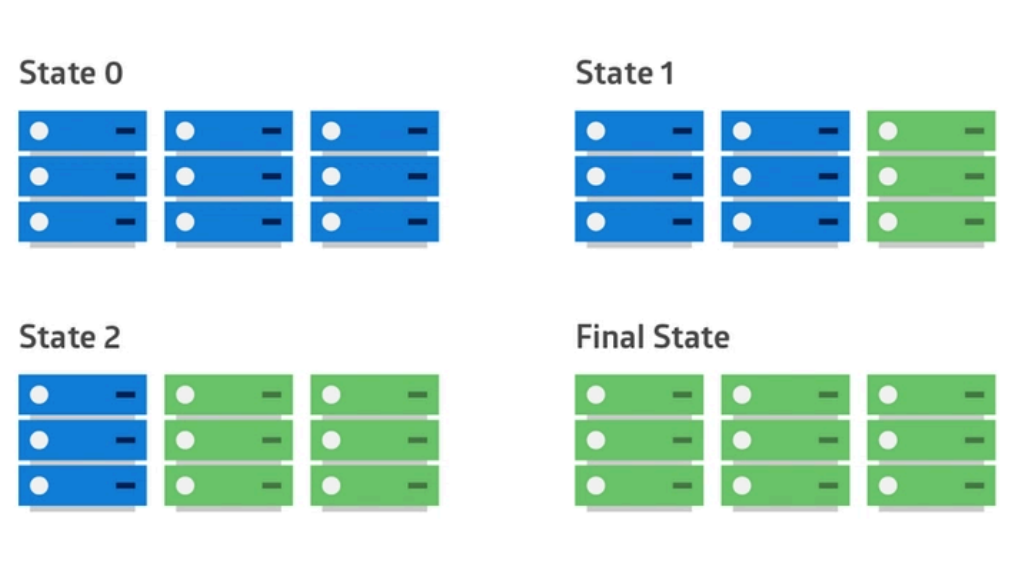
Strengths

- Very safe
- Very fast rollback
- Simple mental model

Weaknesses

- Requires 2 full environments
- Expensive
- Doesn't gradually test with real users

**Rolling Deployment** gradually replaces old instances of the application with new ones across the infrastructure. All users are progressively exposed to the new version as the update spreads.


State 0
State 1
State 2
Final State

When to choose it

- Infrastructure cost must be minimized
- The application is backward-compatible
- You don't need full environment duplication

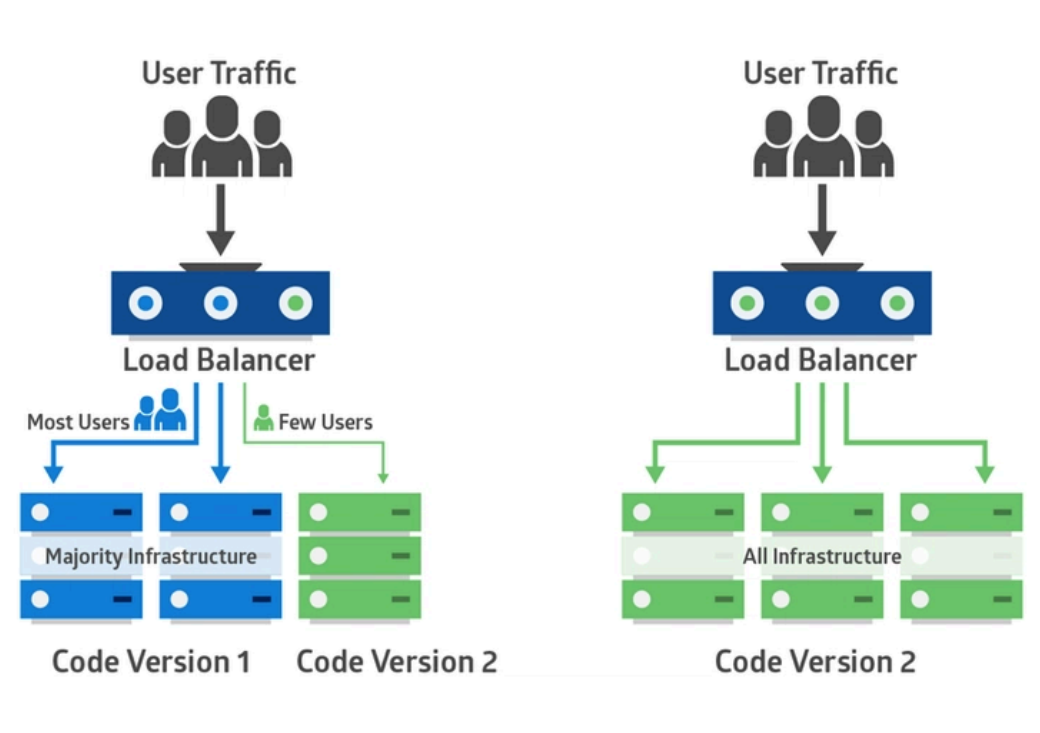
Strengths

- No duplicate environment needed (Cost efficient)
- Built-in support in most orchestrators

Weaknesses

- Slower rollback
- Old and new versions coexist temporarily
- Risk if backward compatibility is broken

**Canary Deployment** starts by releasing the new version to a small subset of users. If no issues are observed, the rollout continues to a larger user base.


User Traffic
Load Balancer
Most Users
Few Users
Majority Infrastructure
Code Version 1
Code Version 2
User Traffic
Load Balancer
All Infrastructure
Code Version 2

When to choose it

- You want **real-user validation**
- You have strong **monitoring and observability**
- You can handle partial failures
- You want controlled risk

Strengths

- Gradual risk exposure
- Data-driven validation
- Can combine with feature flags

Weaknesses

- Requires mature monitoring
- More complex traffic management

## The DevSecOps model

### Motivation

**Application security has traditionally been handled after development is completed**, often by a team separate from both the development and operations teams. This approach slowed down development and delayed response times to security issues.

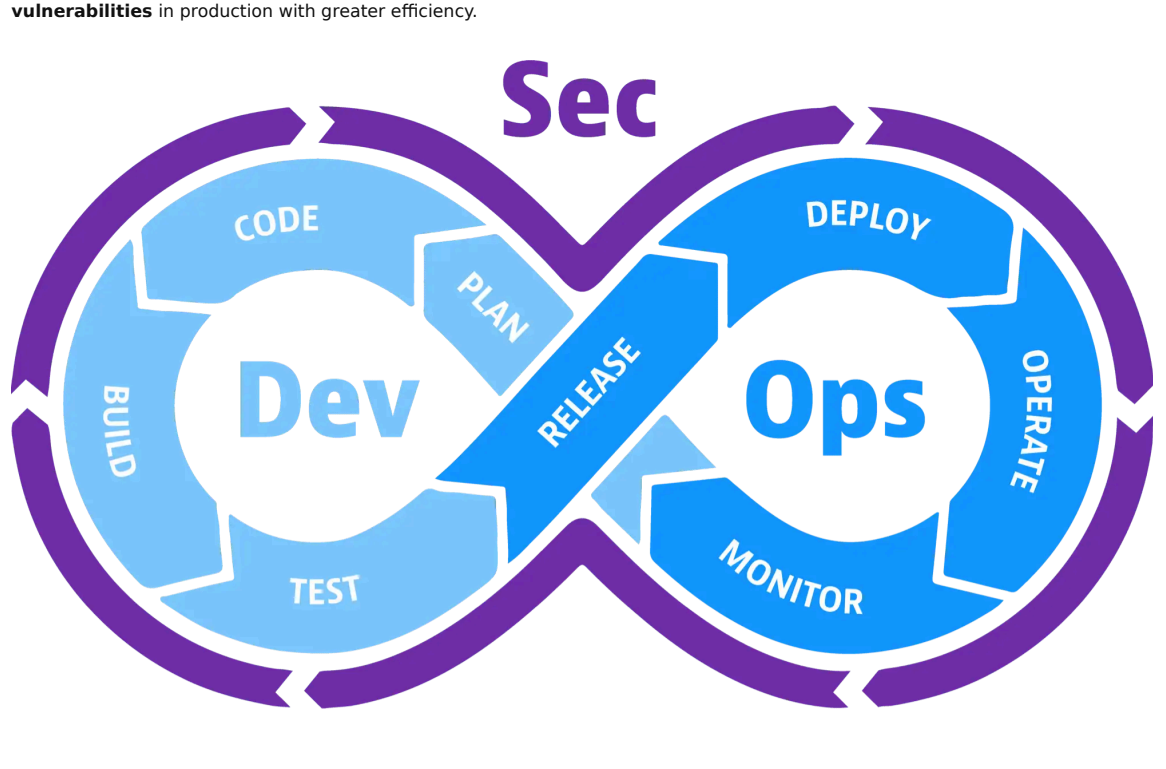
Furthermore, **security tools have typically operated in isolation**, with each application security test focusing only on a single application, often just examining its source code. This made it difficult to gain an **organization-wide view of security concerns** or to assess software risks in the context of the production environment.

### Definition

By integrating application security into a unified DevSecOps process—from the initial design phase to implementation—organizations can align development, security, and operations. **This approach ensures that security is continuously prioritized throughout the software lifecycle**, resulting in a more efficient, collaborative, and secure process for building and maintaining applications.

The primary goal of **DevSecOps** is to embed security practices into the continuous integration and continuous delivery (CI/CD) pipeline across both **pre-production** (development, testing, staging) and **production** (operations) environments.

By adopting DevSecOps, teams can **release higher-quality software more quickly** and **detect and respond to software vulnerabilities** in production with greater efficiency.


Sec
CODE
PLAN
BUILD
Dev
TEST
RELEASE
DEPLOY
Ops
OPERATE
MONITOR

### Challenges

One of the primary challenges in implementing DevSecOps is addressing the people and culture aspect. It often **requires retraining DevOps teams to understand security best practices** and effectively use new security tools. This cultural shift is essential for embedding security into the development process from the outset.

Another key challenge is **selecting the right security tools and integrating them seamlessly into your DevOps workflow**. The more automated and well-integrated your DevSecOps tooling is within the CI/CD pipeline, the less training and cultural adjustment will be needed. This seamless integration ensures that security measures are applied continuously and without friction.

### Key Performance Indicators (KPIs)

1. **Mean Time to Detect (MTTD) and Mean Time to Recover (MTTR)**
   - **MTTD**: This metric measures the average time taken to detect security incidents or vulnerabilities. It indicates the effectiveness of security monitoring, detection systems, and incident response processes in identifying potential threats.
   - **MTTR**: This metric measures the average time required to recover security incidents after they are detected. It reflects the efficiency of incident response, patch management, and vulnerability resolution processes.
2. **Number of Security Vulnerabilities**
   This metric quantifies the total number of vulnerabilities identified during the development cycle. It helps track trends in the identification, remediation, and resolution of security flaws, ensuring that they are addressed promptly before deployment.
3. **Code Review Findings**
   This metric tracks the number and severity of security issues discovered during code reviews. It reflects the effectiveness of secure coding practices, the use of code analysis tools, and developer awareness in identifying and addressing code-level vulnerabilities.
4. **Deployment Frequency**
   This metric measures how frequently software deployments are made to production. It indicates how seamlessly security practices are integrated into the deployment pipeline and highlights the ability to deliver secure software at a fast pace while maintaining quality.

### Key Tools

| Category | Tool Examples | Purpose / Use |
|---|---|---|
| **Container & Image Scanning** | **Trivy**, Clair | Scan container images for vulnerabilities in OS packages and application dependencies. |
| **Static Code Analysis (SAST)** | **Semgrep**, SonarQube | Detect security vulnerabilities in source code; Semgrep allows **custom pattern-based scans** for security issues. |
| **Dependency & Open Source Scanning (SCA)** | **Snyk**, WhiteSource | Find and fix vulnerabilities in third-party libraries. |
| **Secrets & Credential Scanning** | **GitGuardian**, TruffleHog | Detect exposed secrets in code or repositories; GitGuardian specializes in **real-time secret detection in Git repos**. |
| **Code Linting / Quality** | **Super-Linter**, ESLint | Enforce code quality and basic security rules across multiple languages. |
| **Dynamic Application Security Testing (DAST)** | **OWASP ZAP**, Burp Suite | Test running applications for runtime vulnerabilities. |
| **Infrastructure as Code (IaC) Security** | **Checkov**, tfsec | Analyze Terraform, CloudFormation, or Kubernetes manifests for misconfigurations. |
| **Runtime / Container Security** | **Falco**, Aqua Security | Monitor live containers and cloud workloads for suspicious activity. |

## Resources

- DevOps at Netflix
- How Netflix Uses Java - 2025 Edition